

SQL Injection Attack: How It Works and 4 Preventive Measures

What Is an SQL Injection Attack?

SQL Injection is a type of cyber attack where malicious code is inserted into an SQL statement, thereby manipulating the execution of the statement to gain unauthorized access to sensitive data or perform malicious actions.

It exploits vulnerabilities in the application's input validation process, allowing the attacker to execute arbitrary SQL commands and manipulate the database. This can result in data theft, modification, or destruction, potentially causing significant harm to an organization.

This is part of a series of articles about [cybersecurity attacks](#).

How Does a SQL Injection Attack Work?

An SQL injection attack works by exploiting vulnerabilities in the application's input validation process. Here's how it typically works:

- The attacker crafts malicious input, such as through a web form or URL parameter.
- The application takes this input and executes it as part of an SQL query to a database.
- If the application does not properly validate or sanitize the input, the attacker can manipulate the query to perform unintended actions.
- The attacker can use the manipulated query to access sensitive data, modify data, or delete data, depending on the level of privileges the application has on the database.

For example, consider a search form that takes a user's input and executes an SQL query to retrieve data from a database based on the user's search criteria. If the application does not properly validate or sanitize the user's input, an attacker could enter malicious input that manipulates the SQL query and allows them to access sensitive data or modify the database.

Examples of SQL Injection Attacks

Java SQL Injection

Consider the following code snippet in Java, which is vulnerable to SQL injection:

```
String query = "SELECT * FROM users WHERE username='" +  
request.getParameter("username") + "' AND password='" +  
request.getParameter("password") + "'";
```

```
ResultSet resultSet = statement.executeQuery(query);
```

If an attacker inputs the following as the username parameter:

```
' OR '1'='1
```

The resulting query will look like this:

```
SELECT * FROM users WHERE username=' ' OR '1'='1' AND password=''
```

The attacker has successfully bypassed authentication and will be able to access sensitive information.

A safe version of the code would use parameterized queries and PreparedStatements:

```
String query = "SELECT * FROM users WHERE username=? AND password=?";
```

```
PreparedStatement preparedStatement = connection.prepareStatement(query);
```

```
preparedStatement.setString(1, request.getParameter("username"));
```

```
preparedStatement.setString(2, request.getParameter("password"));
```

```
ResultSet resultSet = preparedStatement.executeQuery();
```

In this example, the user inputs are passed as separate parameters and are properly escaped, preventing any malicious code from being executed.

C# SQL Injection

Here is an example of C# code that constructs and executes an SQL query to search for items based on specified criteria:

```
using (SqlConnection connection = new SqlConnection(connectionString))
```

```

{

    connection.Open();

    string searchTerm = "Apple";

    string sql = "SELECT * FROM Products WHERE ProductName LIKE @SearchTerm";


    using (SqlCommand command = new SqlCommand(sql, connection))
    {

        command.Parameters.AddWithValue("@SearchTerm", "%" + searchTerm + "%");

        using (SqlDataReader reader = command.ExecuteReader())

        {

            while (reader.Read())

            {

                Console.WriteLine(reader["ProductName"] + " | " + reader["Description"] + " | " +
reader["Price"]);

            }

        }

    }

}

```

In this example, the code uses a `SqlConnection` object to connect to a database and open a connection. The search term is stored in a variable `searchTerm`. The SQL query is constructed using the `SqlCommand` class, which takes the SQL string and the `SqlConnection` object as parameters.

The query uses a parameterized query and adds the search term to the `SqlCommand` object using the `AddWithValue` method of the `Parameters` collection. The query is executed using the `ExecuteReader` method, which returns a `SqlDataReader` object. The code then loops through the rows returned by the query and outputs the product name, description, and price for each item.

SQL Injection Attacks Protection and Prevention

While it is difficult to completely prevent SQL injection attacks, the following measures can significantly reduce the risk to your applications.

Prepared Statements

Prepared statements are a method of protecting against SQL injection attacks. They allow the programmer to define a parameterized query in advance, and then supply the data to be used in the query at runtime. The database then separates the data from the query, which helps prevent any malicious input from being interpreted as part of the SQL command.

This is a more secure way of executing dynamic SQL queries compared to concatenating the query string and input data, which is vulnerable to SQL injection. By using prepared statements, the risk of SQL injection can be reduced, as the input data is treated as separate from the SQL query, and any special characters or escape sequences are handled correctly by the database.

Stored Procedures

Stored procedures are pre-compiled sets of SQL statements that are stored in the database and can be called as a single executable unit. When a stored procedure is executed, the database first compiles the procedure and then executes the compiled code. This means that any dynamic SQL statements within the stored procedure are parsed and optimized only once, when the stored procedure is first created, rather than each time the procedure is executed.

Stored procedures provide an additional layer of security against SQL injection, as the SQL statements within the stored procedure are protected from tampering by end-users. Additionally, stored procedures can enforce a specific set of business rules, reducing the risk of rogue SQL statements being executed.

Principle of Least Privilege

The principle of least privilege is a security concept that states that an application should only have the minimum necessary permissions to carry out its intended functions. In the context of SQL injection attacks, this means limiting the permissions of the application when interacting with the database to reduce the risk of an attacker using SQL injection to execute malicious actions.

Applications should be designed to only have access to the resources they need. For example, applications can be configured to execute DML statements but not DDL statements,

or to execute with read-only permissions. This restricted privilege can help prevent an attacker from modifying the database structure or executing rogue SQL statements.

Allowlist Input Validation

Allowlist input validation is a method of protecting against SQL injection attacks by checking the input data to ensure it is valid and allowed. This approach involves defining a set of acceptable inputs, also known as an "allowlist," and only accepting data that meets the criteria in the allowlist. Any input that does not match the allowlist is rejected.

For example, if a search field only allows alphanumeric characters and spaces, any input that contains special characters or escapes sequences would be rejected. This helps prevent malicious code from being interpreted as part of an SQL statement, and therefore reduces the risk of SQL injection.

Allowlist vs. denylist

Allowlist input validation provides a more secure method of input validation compared to denylist input validation, where a list of unacceptable inputs is defined and all other inputs are accepted. The problem with denylist validation is that attackers can easily bypass the validation by using inputs that are not on the denylist.

By using allowlist input validation, the risk of SQL injection can be reduced, as malicious input that does not match an acceptable value will not be processed by the application. This helps to prevent attackers from inserting malicious code into an SQL statement and executing unauthorized actions.

However, it's important to keep the allowlist up-to-date and comprehensive to ensure that it covers all possible legitimate input values, otherwise it may cause false positive results or prevent legitimate input from being processed.

SQL Injection Attack Prevention with HackerOne

Vulnerability hunting by ethical hackers will find many of the application flaws used to deliver SQL Injection exploits. The HackerOne Attack Resistance platform delivers continuous, proactive application security with immediate access to security experts who approach your attack surface from an adversarial point of view, to find weaknesses before cyber criminals do.

[Learn more about HackerOne](#)

Company

Leadership

Careers

Partners

Newsroom

Contact Us

Knowledge

Center

Application Security

Penetration Testing

Cloud Security

Hacking

Cybersecurity

Attacks

DevSecOps

Resources

Blog

Documentation

Leaderboard

Partner Portal

Resources

Contacted
by a
hacker?

